

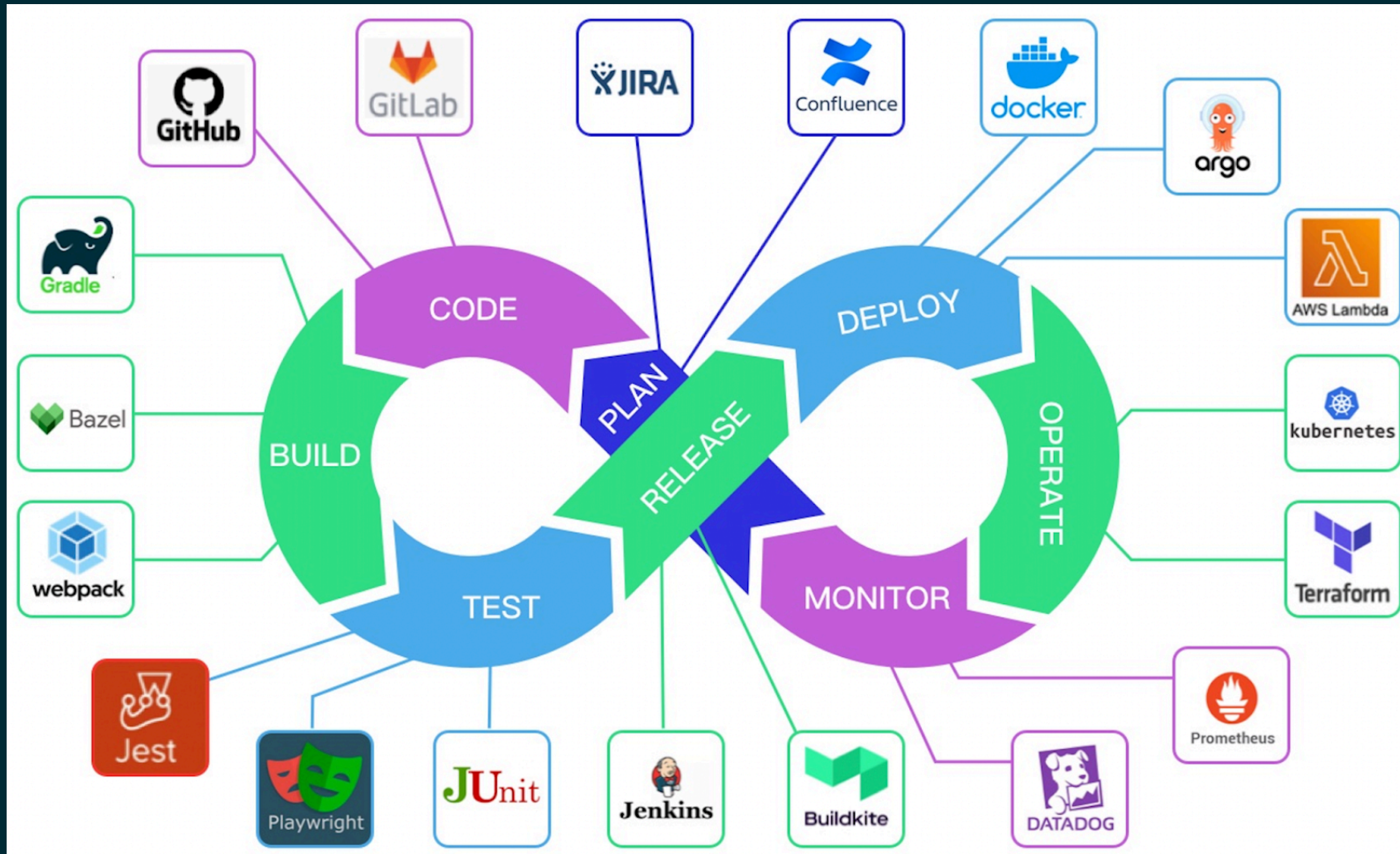
CI/CD

Unit 2 - Continuous Integration & Continuous Deployment

Lucas Bonanni

2024-01-09

CI/CD



¿QUÉ ES CI/CD Y POR QUÉ ES FUNDAMENTAL?

- **Automatización:** Elimina tareas repetitivas y reduce el error humano.
- **Velocidad:** Permite entregar valor al usuario de forma mucho más rápida.
- **Calidad:** Asegura que el código cumple con los estándares mediante pruebas automáticas constantes.
- **Cultura:** Fomenta la responsabilidad compartida (“you build it, you run it”).

CI - CONTINUOUS INTEGRATION

DEFINICIÓN Y OBJETIVOS

Definición: Práctica de automatizar la integración de los cambios de código de varios contribuyentes en un único proyecto de software de forma frecuente (mínimo una vez al día).

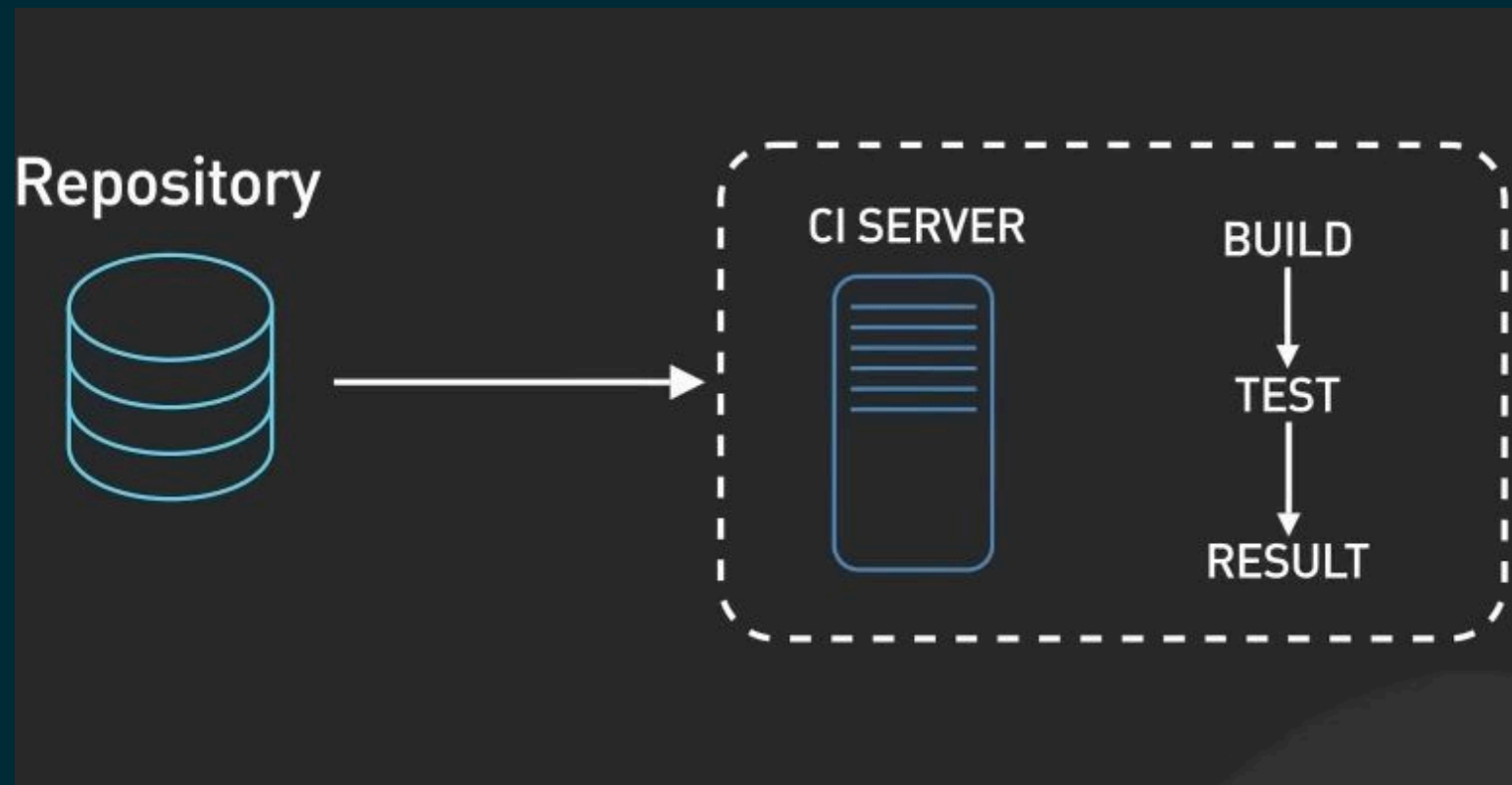
OBJETIVOS PRINCIPALES

1. Detectar errores lo antes posible (Fail Fast).
2. Reducir conflictos de integración (“Integration Hell”).
3. Mantener una base de código siempre estable.

ETAPAS DE CI

Un flujo de CI típico incluye:

1. **Code (Codificación):** El desarrollador sube su código al repositorio.
2. **Build (Construcción):** El sistema compila y prepara los binarios/artefactos.
3. **Test (Pruebas):** Validación automática del código.



CI: CODE

Todo comienza con el control de versiones (SMC).

- **Version Control:** Git es el estándar.
- **Branching Strategies:**
 - *Trunk-based Development:* Ramas cortas, integración diaria.
 - *GitFlow:* Estructura rígida para releases planificados.
- **Code Review:** Pull Requests para asegurar calidad mediante ojos humanos.

Herramientas: GitHub, GitLab, Bitbucket.

CI: BUILD

Transformar el código fuente en algo ejecutable.

- **Gestión de Dependencias:** npm, maven, pip, nuget.
- **Compilación:** Generación de binarios (Java, C#, Go).
- **Contenedorización:** Creación de imágenes Docker.

```
1 # Ejemplo de Build con Docker
2 FROM node:18-alpine
3 WORKDIR /app
4 COPY package*.json ./
5 RUN npm install
6 COPY . .
7 RUN npm run build
```


CI: BUILD

- Github actions
- circleci
- Travis CI
- Jenkins

CI: CODE BUILD

- Webpack
- Gradle
- Bazel

EJEMPLO: GITHUB ACTIONS WORKFLOW

```
1 name: Node.js CI
2 on: [push, pull_request]
3
4 jobs:
5   build-and-test:
6     runs-on: ubuntu-latest
7     steps:
8       - uses: actions/checkout@v3
9       - name: Setup Node
10         uses: actions/setup-node@v3
11         with: { node-version: '18' }
12       - run: npm install
13       - run: npm test
14       - run: npm run build
```

CI: TEST (PRUEBAS AUTOMATIZADAS)

Nuestra red de seguridad.

- **Unit Tests:** Prueban la lógica interna de funciones.
- **Integration Tests:** ¿Cómo hablan mis módulos entre sí?
- **Static Analysis (Linting):** Calidad de código y estilo.
- **Security (SAST):** Análisis estático de vulnerabilidades.

Herramientas: Jest, JUnit, PyTest, SonarQube, Snyk.

CD - CONTINUOUS DELIVERY & DEPLOYMENT

DELIVERY VS DEPLOYMENT: LA DIFERENCIA CLAVE

CONTINUOUS DELIVERY (ENTREGA CONTINUA)

El código está siempre listo para ser desplegado, pero el paso final a producción se decide manualmente (un botón de “Aprobar”).

CONTINUOUS DEPLOYMENT (DESPLIEGUE CONTINUO)

Todo el proceso es automático. Si el código pasa las pruebas, llega a producción automáticamente.

ETAPAS DE CD: RELEASE & DEPLOYMENT

- **Release:** Versionado (SemVer), creación de notas de lanzamiento, almacenamiento en registros (Artifact Registry).
- **Deployment:** El acto de poner el código en el servidor/cloud.

Estrategias de Despliegue: - **Blue/Green:** Cero tiempo de inactividad, fácil rollback. - **Canary:** Probar con pocos usuarios reales antes del despliegue total.

CONTINUOUS DEPLOYMENT

¿Todas las empresas tienen procesos de CD automatizados?

DEPLOYMENT STRATEGIES

- Blue/Green Deployment
- Rolling deployment
- Depliegue progresivo
- Rollback automático

DEPLOYMENT TOOLS

- Docker
- Argo
- Lambdas

VERSIONADO: BETAS

- Retroalimentación temprana de los usuarios
- Validación de características
- Iteración rápida y mejora continua
- Reducir el riesgo de lanzamiento
- Promoción y generación de interés

OPERATE & MONITORING

El ciclo no termina cuando el código “está vivo”.

- **Operate:** Infraestructura como Código (IaC), escalado automático, gestión de secretos.
- **Monitoring:** Observabilidad (Logs, Métricas, Traces).
- **Feedback Loop:** Los datos de producción informan la planificación futura.

Herramientas: Kubernetes, Prometheus, Grafana, ELK, Terraform.

OPERATE

- Kubernetes
- Terraform

MONITORING

- Datadog
- new relic
- Grafana
- Prometheus

RESUMEN: CI/CD Y DEVOPS

CAMS EN CI/CD

- **Culture:** Colaboración y transparencia en el proceso de entrega.
- **Automation:** El core del pipeline.
- **Measurement:** Pruebas automáticas y monitoreo de performance.
- **Sharing:** Repositorios compartidos y visibilidad total del estado de los despliegues.

“Si duele, hazlo más seguido” - Si integrar código es difícil, hazlo cada hora, no cada mes. La frecuencia reduce la

RECURSOS Y PRÓXIMA CLASE

- [Martin Fowler: Continuous Integration](#)
- [The DevOps Handbook \(Gene Kim, Jez Humble\)](#)

Próxima Clase: Full CI/CD process hands-on -

Configuración de GitHub Actions desde cero. - Despliegue automático a un Cloud Provider. - Simulación de fallos y rollbacks.